

Lecture Notes

Large Language Models (LLMs) and Chatbot Functionality#

*** (Welcome! Think of me as your guide through this complex topic. We are going beyond simple definitions; we are understanding the **machinery** that makes modern AI possible. Don't worry if some terms sound intimidating—we will break them down using analogies.) ***

What is it?

****Simple Definition:**** An LLM is a highly sophisticated mathematical function designed to predict what word should come next in any given sequence of text. It doesn't "think" or "understand" in the human sense; it is an incredibly advanced pattern recognition machine that predicts probability.

****Technical Explanation:**** At its core, an LLM takes a piece of input text (the **prompt**) and assigns a ****probability score**** to every single word in its massive vocabulary. The output isn't just one answer; it's a full probability distribution showing how likely every possible next word is. A chatbot simply uses this prediction function repeatedly—predicting the first word, then predicting the second word based on the input **plus** the first predicted word, and so on, until the dialogue feels complete.

Why do we need it?

Before LLMs, AI systems were often "rule-based." If you wanted a chatbot to know about banking, you had to program thousands of specific rules (e.g., IF user says 'balance' THEN respond with X). This was brittle and couldn't handle unexpected phrasing.

LLMs solved this problem by shifting the focus from ****rules**** to ****patterns****.

- **Handling Ambiguity:**** LLMs don't need explicit rules for everything. Because they are trained on trillions of words, they have internalized statistical patterns that allow them to generate coherent text even when faced with novel or ambiguous inputs (e.g., "The bank was steep" vs. "I went to the river bank").
- **Scale and Scope:**** They can handle vast amounts of knowledge simply because their training data encompasses so much human writing, making them incredibly versatile assistants.

How does it work?

Understanding LLMs requires looking at three major stages: The Architecture, The Training Process, and The Output Generation.

■ Part 1: The Core Architecture (The Transformer)

The breakthrough that made modern LLMs possible was the ****Transformer**** architecture (introduced by Google).

* ****Old Way:**** Older models processed text sequentially—like reading a sentence word-by-word from left to right, which is slow.

* **New Way (Parallelism):** Transformers process all words in an input simultaneously (**in parallel**). This massive speed boost allowed for the training on truly enormous datasets.

■ Key Component: The Attention Mechanism

This is the "magic" of the Transformer. Think of it like a spotlight that focuses attention when you read.

* When reading the sentence: "The bank near the river was steep."

* When the model processes the word "bank," the Attention mechanism doesn't treat it in isolation. It looks at every other word ("river," "steep") and assigns a weight (or score) to how relevant they are to defining "bank."

* The attention allows the meaning of "bank" to shift contextually, knowing that because "river" is nearby, it must mean a river bank, not a financial institution.

■ Part 2: The Training Process (How LLMs Learn)

LLM training is complex and happens in stages:

1. Pre-training (The Massive Homework):

* **Goal:** To teach the model general grammar, facts, and language patterns from raw data.

* **Process:** The model is fed massive amounts of internet text. It is trained on a simple task: **predicting the next word.**

* **Example:** Input: "The capital of France is..." Model must predict: "...Paris."

* **Mechanism (Backpropagation):** If the model predicts "London," but the true answer is "Paris," an algorithm called **backpropagation** calculates the error. It then tweaks billions of internal settings (**parameters/weights**) slightly so that next time, it will be more likely to choose "Paris" and less likely to choose "London."

* **Result:** The model learns general knowledge but might sound random or nonsensical if not guided.

2. Fine-tuning (The Coaching):

* **Goal:** To make the raw predictor useful, helpful, and safe—like turning a brilliant student into a polite assistant.

* **Method:** Reinforcement Learning with Human Feedback (RLHF).

* Human reviewers act as "trainers." They interact with the model and flag bad answers ("This is unhelpful," or "This violates policy").

* The model's parameters are then tweaked again, specifically to make it more likely to give human-preferred, helpful responses.

■ Part 3: Generating an Answer (Inference)

When you type a prompt and hit send, the following happens:

1. **Encoding:** Your prompt is converted into numerical vectors that the Transformer can process.
2. **Contextualization:** The Attention mechanism runs, refining every word's meaning based on all other words in your prompt.
3. **Prediction Loop:** The model predicts a probability distribution for the next word.

4. **Sampling (The Natural Touch):** Instead of always picking the single most probable word (which can sound robotic), the system samples from this probability list, sometimes selecting slightly less likely but still plausible words at random. This randomness is what makes the output feel natural and human-like.

Example: The Ambiguity Test

Imagine you are training an LLM on text that contains both articles about finance and geography.

Prompt: "We visited a large _____ by the water."

1. **The Model's Internal Process:** The model sees the word "by" and "water." It activates its Attention mechanism.
2. **Contextual Weighting:** The presence of "river," "shore," or "ocean" (if they were in the surrounding text) increases the probability weight for words like **"bank"** (meaning river edge). Conversely, if the prompt was about money, the weights would shift to mean a financial institution.
3. **Prediction:** The model predicts that the most probable word is **"river bank"**, making the sentence coherent and contextually accurate.

Important Points (Key Vocabulary)

* **Parameters/Weights:** These are the billions of continuous numerical values inside the model. They determine how strongly one piece of input affects another. Think of them as the "knobs" or "dials" that define the model's behavior.

* **Parallel Processing:** The ability to process many pieces of data (words/vectors) at the same time, which is enabled by GPUs and the Transformer architecture. This is crucial for scale.

* **Emergent Behavior:** This refers to capabilities or behaviors that appear in a very large model **without** being explicitly programmed. For example, while you might not program "the ability to write poetry," it emerges as a capability because of the sheer scale of training data and parameters.

* **Deterministic vs. Stochastic:** The underlying math is **deterministic** (given the same input and settings, it always calculates the

The Mechanics of LLM Prediction: Probability and Autocompletion

■ What is it?

At its simplest level, a Large Language Model (LLM) is an extremely sophisticated **statistical prediction machine**. It does not "think" or "know" facts in the human sense; rather, its entire function is to calculate the probability of what word should come next, given all the words that came before it.

Think of it like this: If you give the model a prompt (e.g., "The capital of France is"), the LLM doesn't look up an answer in a database. Instead, it processes those tokens and determines that based on its training data, the word **"Paris"** has the highest probability of appearing next.

Key Concept: Probability Distribution

When the LLM makes a prediction, it does not commit to just one word. It generates a **probability distribution**. This means it assigns a score (a probability) to **every single word** in its massive vocabulary.

- * **High Probability:** Words like "Paris" might get 80% chance.

- * **Medium Probability:** Related words like "France" or "city" might get 5% chance each.

- * **Low Probability:** Completely random words like "flibbertigibbet" might get 0.001% chance.

The model then selects the next word based on these probabilities, often with a degree of randomness (controlled by parameters like **temperature**) to make the text sound more natural and less repetitive. This process repeats, predicting one word after another, until the response is complete.

■ Why do we need it?

Before LLMs, AI systems were generally based on **keyword matching** or rigid rules (e.g., "If user says X, output Y"). These systems were brittle; if you phrased a question slightly differently, they would fail.

LLMs revolutionized this because they allow the machine to understand the **context** and **relationship** between words, not just the words themselves.

1. **Contextual Understanding:** They can handle ambiguity. If you write "The bank of the river," the model knows that "bank" refers to geography, not a financial institution, because it analyzes the surrounding word ("river").
2. **Fluency and Coherence:** Because they predict words based on massive amounts of human writing (the internet), their output is incredibly natural, coherent, and fluent—it reads like something a person actually wrote.
3. **Scalability:** They can perform tasks ranging from summarizing complex articles to translating languages, all using the same core mechanism: predicting the next most logical word.

■■ How does it work? (The Mechanics Breakdown)

Understanding how an LLM works requires understanding three major stages: Training Data, The Architecture, and The Prediction Loop.

1. Training Phase I: Pre-training (Learning Grammar & Facts)

- * **Goal:** To teach the model basic grammar, syntax, and general world knowledge by forcing it to fill in blanks across trillions of words.

- * **Mechanism:** **Backpropagation.**

- * The model is fed a massive piece of text (e.g., "Dogs are mammals because they...").

- * It predicts the next word ("breathe").

- * It compares its prediction to the **true** word in the training data ("live").

* If it was wrong, an algorithm called **Backpropagation** adjusts billions of internal dials (called **parameters** or **weights**) slightly. These adjustments make the model less likely to choose incorrect words and more likely to choose the correct words next time.

* **Result:** The model learns patterns at a staggering scale.

2. Training Phase II: Refinement (Making it Helpful)

* **Goal:** To shift the model from simply "autocompleting random text" to being a useful, safe, and helpful assistant.

* **Mechanism:** **Reinforcement Learning with Human Feedback (RLHF).**

* Human reviewers act as trainers. They flag bad answers ("This is biased," or "This answer is unhelpful").

* The model's parameters are adjusted again to favor the types of responses humans prefer, making it better at following instructions and staying on topic.

3. The Architecture: Transformers and Attention (Processing Context)

Before 2017, models processed text word-by-word (slowly). The **Transformer** architecture changed everything by allowing the model to process all words in a prompt *simultaneously* (*in parallel*).

* **Step A: Encoding (The Numbers):** Every word must be converted into a long list of numbers (a vector or embedding). This is necessary because computation only works with continuous values, not letters.

* **Step B: Attention Mechanism (The Context Whisperer):** This is the most critical part. Instead of treating every number equally, the attention mechanism allows each word's numerical representation to "talk" to and weigh all other words in the input context.

* **Analogy:** Imagine a group discussion where everyone listens to everyone else. When you hear the word "bank," your brain doesn't just see that single word; it pays attention to surrounding words like "river" or "money." Attention does this mathematically, refining the meaning of 'bank' based on its neighbors.

* **Step C: Prediction:** After the context has been fully refined by the attention mechanism and other layers (like the feed-forward network), a final function takes this enriched understanding and outputs the full probability distribution for the next word.

■ Example Time! (The "Riverbank" Test)

Imagine you are building an LLM to understand context:

Scenario: You want the model to correctly distinguish between two uses of the word "bank."

1. **Input Prompt A:** "We sat by the **bank** and watched the water flow..."

* **Attention's Job:** The attention mechanism notices the words "sat," "by," and "water." It assigns a high weight (attention) between these words and the target word "bank."

* **Result:** The model adjusts its internal representation of "bank" to mean **river edge**.

2. **Input Prompt B:** "I need to deposit money at the local **bank**."

Attention's Job: It notices "deposit," "money," and "local." These words heavily influence the meaning of "bank."

Result: The model adjusts its internal representation of "bank" to mean **financial institution**.

The LLM doesn't just remember two definitions; it **calculates** which definition is most mathematically probable based on the context you provide.

■ Important Points (Must-Know Vocabulary)

| Term | Simple Definition | Why It Matters |

| :--- | :--- | :--- |

| **Parameters/Weights** | The billions of internal dials or knobs that determine the model's behavior. They are adjusted during training. | These values **are** the knowledge and capability of the LLM. More parameters often mean more complexity (and better performance). |

Model Training Process: Pre-training and Parameters

What is it?

At its heart, a Large Language Model (LLM) is not a database of facts; **it is a sophisticated mathematical function designed to predict the next word in any given sequence of text.**

Think of an LLM as an extremely advanced auto-complete feature. When you type "The capital of France is...", the model doesn't **know** the answer; it calculates that based on all the billions of texts it has read, the word "Paris" has the highest probability of appearing next.

Parameters (or Weights):

These are the core variables or values within the LLM's mathematical structure. You can think of them as **billions of tiny knobs or dials**. Every single dial determines how strongly one piece of information influences another piece of information during prediction. The model's entire behavior—its grammar, its knowledge, its style—is encoded in the specific settings of these parameters.

Why do we need it?

We need this complex training process for two main reasons: **Scale** and **Context**.

1. **To Achieve Fluency (The Scale Problem):** Human language is incredibly vast, diverse, and nuanced. To predict a word accurately, the model must have seen enough examples of how words relate to each other across countless topics (history, science, poetry, conversation). Training on massive amounts of data (trillions of words) allows it to capture these complex patterns.

2. **To Understand Context (The Meaning Problem):** Language is ambiguous. The meaning of a word often changes based on the surrounding words (e.g., "bank" could mean a financial institution or the side of a river). We need specialized mechanisms like **Attention** to allow the model to weigh different context clues simultaneously, ensuring it picks the most appropriate meaning.

How does it work?

The training process is not a single step; it's a multi-stage pipeline:

1. Pre-training (Prediction by Guessing)

* **Goal:** To learn general grammar and predict missing words based on vast, raw data (like scraping the entire internet).

* **Process:** The model is fed huge chunks of text. It is given all but the last word, and it must guess what the last word is.

* **Example:** Input: "The cat sat on the ______ ." Model must predict "mat" or "rug".

* **Mechanism (Backpropagation):** When the model guesses incorrectly, an algorithm called **backpropagation** calculates *how wrong* it was. It then slightly tweaks every single parameter (every dial) in the model to make that mistake less likely next time and more likely to choose the correct word. This process is repeated trillions of times.

* **Result:** The model becomes excellent at predicting text patterns, but it might still be random, toxic, or unhelpful because it only learned *what* words follow each other, not *how* a good assistant should behave.

2. Fine-Tuning and Alignment (Making it Helpful)

* **Goal:** To take the raw knowledge from pre-training and align it with human preferences, making it safe, helpful, and conversational.

* **Process:** Reinforcement Learning with Human Feedback (RLHF):

1. The model generates several possible answers to a prompt.
2. Human reviewers rank these answers (e.g., Answer A is best, Answer C is worst).
3. This human ranking data is used to train a separate "Reward Model."
4. The main LLM then trains itself using this Reward Model, learning to generate responses that maximize the score given by humans.

3. The Architecture: Transformers and Attention (The Engine)

* **Problem:** Older models processed text word-by-word (slowly).

* **Solution:** The Transformer: This architecture allows the model to process an entire sequence of words *in parallel*. It "soaks it all in at once."

* **Attention Mechanism:** This is the magic ingredient. When processing a sentence, Attention doesn't treat every word equally. For example, if the sentence is "The river bank was deep," when the model processes the word **"bank"**, the attention mechanism forces that word to pay special *attention* to the word **"river"**. This context clue tells the model that it must use the definition of a geographical bank, not a financial one.

Example

Imagine you are training a student (the LLM) on English grammar using flashcards:

| Component | Analogy | Function in LLMs |

| :--- | :--- | :--- |

| **The Student** | The Large Language Model | Processes input and predicts output. |

| **Flashcard Input** | "I went to the [...]" | The prompt (the words given). |

| **Correct Answer** | "store" | The true next word in the training data. |

| **Parameters/Weights** | The Student's Brain Connections | Millions of dials that adjust the strength of connections between concepts (e.g., changing the connection strength between 'I,' 'went,' and 'to'). |

| **Backpropagation** | Grading Test Papers | Comparing the model's guess to the correct answer, calculating the error, and adjusting all internal "connections" (parameters) slightly to improve for next time. |

| **Attention** | Highlighting Key Words | When reading a sentence, you instinctively know which words are most important for meaning. Attention does this mathematically by giving more weight/focus to context clues. |

Important Points

* **Scale is everything:** The sheer number of parameters (hundreds of billions) and the massive data corpus required are what define "Large."

* **Prediction, not knowledge:** LLMs do not *understand* in a human sense; they are highly sophisticated pattern matchers that assign probabilities.

* **Emergent Behavior:** Because of the huge number of parameters, the model develops capabilities (like reasoning or code generation) that were never explicitly programmed—they simply *emerge* from the training process.

* **Parallelism is key:** The Transformer architecture solved the speed problem by allowing all parts of the input text to be processed simultaneously, making massive training feasible.

Common Mistakes

1. **Mistake: Thinking LLMs are omniscient.** They only know what they have read and how those words relate statistically. If the data is biased or wrong, the model will repeat that bias or error (this is called "hallucination").

2. **Mistake: Confusing Pre-training with Fine-tuning.** Pre-training gives the model *raw knowledge*. Fine-tuning/RLHF teaches it *manners* and *utility*. A raw pre-trained model might know everything but be rude, unhelpful, or dangerous.

3. **Mistake: Believing parameters are fixed.** Parameters are not static; they are continuously adjusted (tuned) by the backpropagation process based on every single training example.

Interview Questions

1. **Q:** What is the fundamental difference between pre-training and fine-tuning/alignment?

* **A:** Pre-training teaches general linguistic patterns using raw data; Fine-tuning uses human feedback to teach desirable behaviors, safety, and helpfulness.*

2. **Q:** How does the Attention mechanism improve upon sequential processing?

* **A:** It allows the model to process all words simultaneously (in parallel) and, crucially, it assigns weighted importance (attention scores) between every pair of words, allowing it to understand context instantly rather than one word after another.*

3. **Q:** If an LLM is just predicting the next word, how can it appear to "